



U N I V E R S I D A D
COMPLUTENSE
M A D R I D



Trabajo práctico temas 8 a 12

Máster en Ingeniería de Sistemas y Control

Asignatura: Visión por Computador

Curso: 2025/2026

Alumno: Javier Arambarri Calvo

11 de enero de 2026



Autoría

El autor de este documento es Javier Arambarri Calvo, alumno del Máster en Ingeniería de Sistemas y Control y matriculado por la Universidad Complutense de Madrid en el curso académico 2025-2026.

Para que así conste, se firma digitalmente este documento.

Repositorio

El código de la implementación de los métodos expuestos se encuentra en el siguiente repositorio:
https://github.com/arambarricalvoj/vpc_misc_ucm-uned_25-26.



Índice

1. Introducción	5
1.1. Objetivo y motivación del proyecto	5
1.2. Metodología	6
2. Detección de movimiento en imágenes	7
2.1. Técnicas de detección de movimiento	7
2.2. Modelo <i>MOG2</i> para sustracción de fondo	7
2.2.1. Formulación matemática del modelo mejorado <i>MOG2</i>	8
2.2.2. Ejemplo para comprender la formulación matemática	9
2.2.3. Aprendizaje del fondo en <i>MOG2</i>	11
2.2.4. Implementación en <i>OpenCV</i>	12
2.2.5. Resultados	13
2.3. Extracción de regiones y detección de vehículos	14
2.4. Región de interés	15
3. Flujo óptico	17
3.1. Formulación matemática del flujo óptico de Farnebäck	17
3.1.1. Ejemplo ilustrativo del vector de flujo óptico	20
3.2. Seguimiento de objetos	20
3.3. Cálculo de la velocidad de movimiento del objeto	21
3.4. Implementación en <i>OpenCV</i>	24
3.5. Resultados	24
4. Conclusiones	26

Índice de figuras

1.	Sustracción del fondo con <i>Mog2</i>	13
2.	Sustracción ruidosa del fondo con <i>Mog2</i>	13
3.	Sustracción del fondo con <i>Mog2</i> con dos vehículos cercanos.	14
4.	Píxeles aislados o regiones espurias.	15
5.	Problemas de segmentación.	15
6.	Problemas de segmentación solucionados.	15
7.	Problemas de seguimiento (ID 4 se transforma en ID 6).	16
8.	Región de interés seleccionada.	16
9.	Representación multiresolución en pirámide de imágenes [16].	19
10.	Modelo de cámara geométrico ideal <i>pinhole</i> . Fuente: Campus Virtual.	22
11.	Seguimiento de vehículos, cálculo de velocidad y dirección de desplazamiento. . .	25



1. Introducción

Este trabajo práctico se enmarca en el segundo bloque de la asignatura de Visión por Computador del Máster en Ingeniería de Sistemas y Control, correspondiente a los temas 8 a 12. De entre los trabajos propuestos en la guía de la asignatura, este responde a los temas de detección de movimiento en imágenes y detección del flujo óptico.

Se aborda el diseño e implementación de un sistema capaz de detectar y seguir vehículos en una secuencia de vídeo mediante técnicas clásicas de visión por computador. En particular, se emplean métodos basados en sustracción de fondo para la detección de movimiento y técnicas de flujo óptico para estimar el desplazamiento de los objetos a lo largo del tiempo. Estas herramientas permiten analizar dinámicamente la escena y extraer información relevante sobre los vehículos presentes, como su trayectoria o su velocidad relativa.

El trabajo incluye también un proceso de ajuste y depuración destinado a resolver problemas habituales en este tipo de sistemas, como la aparición de detecciones fantasma durante los primeros fotogramas, la fusión de varios vehículos en una única región de píxeles conectados o *blob* debido a la morfología de la escena o la pérdida temporal de objetos causada por oclusiones o variaciones de iluminación.

Además del material disponible en el Campus Virtual, se han utilizado los artículos [1, 2] como referencia durante el desarrollo del proyecto.

1.1. Objetivo y motivación del proyecto

El objetivo principal de este proyecto es desarrollar un sistema capaz de detectar y seguir vehículos en movimiento a partir de una secuencia de imágenes, asignando a cada uno de ellos una identidad persistente y estimando su desplazamiento y velocidad entre fotogramas consecutivos. Para ello se ha empleado el conjunto de datos GRAM-RTM [3], que proporciona secuencias reales de tráfico urbano y de autopista, lo que permite evaluar el comportamiento del sistema en condiciones cercanas a escenarios reales. En este trabajo únicamente se ha utilizado el conjunto de autopista.

Desde un punto de vista científico y técnico, el proyecto se enmarca en el estudio de técnicas clásicas de visión por computador, concretamente en los ámbitos de la sustracción de fondo, la detección de movimiento y el cálculo del flujo óptico. Estas técnicas constituyen la base de numerosos sistemas modernos de monitorización de tráfico, análisis de movilidad y asistencia a la conducción. El desarrollo de un sistema funcional permite comprender de forma práctica las ventajas y limitaciones de estos métodos, así como los retos asociados a su aplicación en entornos reales.

Desde un punto de vista académico, el objetivo del proyecto es poner en práctica los contenidos teóricos estudiados en la asignatura y ser evaluado.

1.2. Metodología

La metodología seguida en este proyecto se basa en la implementación progresiva de un sistema de detección y seguimiento de vehículos, estructurado en varias etapas claramente diferenciadas. Cada una de ellas aborda un componente fundamental del procesamiento de vídeo en visión por computador.

En primer lugar, se aplica un método de sustracción de fondo basado en el algoritmo *MOG2* [4], encargado de identificar las regiones de la imagen que presentan movimiento. Este proceso incluye un periodo inicial de *warm-up* para posibilitar que la técnica aprenda el fondo estático de la escena y evitar así la aparición de detecciones fantasma en los primeros fotogramas. Tras la sustracción de fondo, se aplican operaciones morfológicas y una máscara de región de interés (*ROI*) para eliminar ruido y restringir el análisis a la zona útil de la carretera.

Esta *ROI* se ha seleccionado previamente mediante una interfaz gráfica diseñada para ello.

A continuación, se extraen los contornos de las regiones detectadas y se generan cajas delimitadoras (*bounding boxes*) para cada posible vehículo. Estas detecciones se filtran mediante criterios geométricos, como el área mínima o la relación de aspecto, con el fin de descartar objetos irrelevantes o artefactos producidos por el sustractor de fondo.

Para el seguimiento temporal de los vehículos, se implementa un sistema de *tracking* basado en la asociación de detecciones mediante distancia euclídea entre centroides. Cada vehículo recibe un identificador único que se mantiene en el tiempo mientras el objeto siga siendo visible. Además, se calcula el flujo óptico de Farnebäck entre fotogramas consecutivos [5], lo que permite estimar el desplazamiento medio de cada vehículo dentro de su región y obtener una medida relativa de su velocidad en píxeles por fotograma.

Durante el desarrollo se realiza un proceso de evaluación del sistema, analizando su comportamiento en diferentes situaciones: tráfico denso, vehículos lejanos u oclusiones parciales. Este análisis permite identificar las limitaciones del enfoque y valorar la eficacia de las soluciones implementadas para mejorar la estabilidad y la precisión del sistema.

2. Detección de movimiento en imágenes

La detección de movimiento constituye el primer paso fundamental en un sistema de análisis de tráfico basado en visión por computador. Su objetivo es identificar las regiones de la imagen que contienen objetos en movimiento, separándolas del fondo estático.

2.1. Técnicas de detección de movimiento

A lo largo de la literatura se han propuesto diversas técnicas para la detección de movimiento, entre las que destacan [6]:

- **Diferencias entre fotogramas:** consiste en restar dos imágenes consecutivas y umbralizar el resultado. Es un método sencillo y eficiente, pero muy sensible al ruido y a variaciones de iluminación.
- **Modelos de fondo estático:** se construye una imagen de referencia del fondo y se compara cada nuevo fotograma con ella. Aunque mejora la estabilidad, requiere mecanismos de actualización para adaptarse a cambios en la escena.
- **Modelos probabilísticos del fondo:** entre ellos destacan los modelos de mezcla de gaussianas (*GMM*), introducidos por Stauffer y Grimson [4]. Estos modelos representan cada píxel como una combinación de distribuciones gaussianas que se actualizan dinámicamente, permitiendo manejar variaciones de iluminación, sombras y movimientos periódicos del fondo.
- **Métodos basados en aprendizaje profundo:** como segmentación semántica o redes de detección de objetos. Aunque ofrecen gran precisión, requieren el uso de técnicas de inteligencia artificial y por tanto, un entrenamiento previo y un coste computacional elevado, por lo que no se han considerado en este proyecto.

Entre todas estas alternativas, sin atender a los métodos de inteligencia artificial, los modelos *GMM* han demostrado ser especialmente adecuados para escenarios de tráfico, donde el fondo presenta variaciones graduales y los objetos en movimiento tienen una apariencia bien definida. Estos modelos constituyen la base de muchos algoritmos de sustracción de fondo modernos, por ejemplo, de *MOG2* [7, 8], implementado en *OpenCV* [9] y seleccionado para este trabajo.

2.2. Modelo *MOG2* para sustracción de fondo

Se trata de una implementación mejorada [7, 8] del modelo de mezcla de gaussianas (*GMM*) propuesto originalmente en [4].

En el enfoque clásico, cada píxel se modela mediante un conjunto de gaussianas que representan las posibles apariencias del fondo. Cuando un nuevo valor de píxel no se ajusta a ninguna de las gaussianas dominantes, se clasifica como primer plano, es decir, como un objeto en movimiento.

Por su parte, en el enfoque mejorado se permite ajustar de forma automática el número de gaussianas necesarias para modelar cada píxel, mejorando la estabilidad del método ante cambios de iluminación, ruido y variaciones dinámicas en la escena.

En este proyecto, *MOG2* se utiliza para generar una máscara binaria que distingue entre fondo y objetos en movimiento. A partir de esta máscara, se aplican operaciones de umbralización, filtrado y morfología para eliminar ruido y mejorar la coherencia espacial de las regiones detectadas. A continuación se presenta una descripción matemática del funcionamiento del modelo *MOG2*, basado en el enfoque de mezcla de gaussianas por píxel.

2.2.1. Formulación matemática del modelo mejorado *MOG2*

El modelo *MOG2* mantiene la misma formulación probabilística básica del modelo *GMM* original, pero introduce un mecanismo adaptativo para estimar el número óptimo de gaussianas por píxel. En lugar de fijar un número K constante, el algoritmo ajusta dinámicamente cuántos componentes son necesarios para modelar la distribución del píxel.

Cada píxel se modela como una probabilidad:

$$p(x_t) = \sum_{i=1}^{K_t} w_{i,t} \mathcal{N}(x_t \mid \mu_{i,t}, \sigma_{i,t}^2),$$

donde:

- x_t es el valor del píxel en el instante temporal t . Puede ser un valor de intensidad (en imágenes en escala de grises) o un vector de tres componentes (en imágenes en color).
- K_t es el número de gaussianas utilizadas para modelar el píxel en el instante t . A diferencia del modelo original, en *MOG2* este número puede variar dinámicamente en función de la complejidad de la distribución del píxel.
- $w_{i,t}$ es el peso de la i -ésima gaussiana en el instante t . Representa la probabilidad de que el píxel pertenezca a ese componente del modelo. Los pesos cumplen:

$$\sum_{i=1}^{K_t} w_{i,t} = 1.$$

- $\mathcal{N}(x_t \mid \mu_{i,t}, \sigma_{i,t}^2)$ es la densidad de probabilidad de una distribución normal unidimensional evaluada en x_t , con media $\mu_{i,t}$ y varianza $\sigma_{i,t}^2$. En el caso de imágenes en color, la gaussiana es multivariante en tres dimensiones.
- $\mu_{i,t}$ es la media de la i -ésima gaussiana en el instante t . Representa el valor típico del píxel cuando pertenece a ese componente del modelo.
- $\sigma_{i,t}^2$ es la varianza asociada a la gaussiana i en el instante t . Controla la dispersión de los valores del píxel alrededor de la media.

Actualización adaptativa de los pesos Zivkovic propone actualizar los pesos mediante:

$$w_{i,t} = (1 - \alpha)w_{i,t-1} + \alpha M_{i,t},$$

donde:

$$M_{i,t} = \begin{cases} 1, & \text{si la gaussiana } i \text{ coincide con } x_t, \\ 0, & \text{en caso contrario.} \end{cases}$$

Actualización de media y varianza Si el píxel coincide con la gaussiana i , sus parámetros se actualizan mediante:

$$\mu_{i,t} = (1 - \rho_t)\mu_{i,t-1} + \rho_t x_t, \quad \sigma_{i,t}^2 = (1 - \rho_t)\sigma_{i,t-1}^2 + \rho_t(x_t - \mu_{i,t})^2,$$

donde ρ_t es un factor de aprendizaje adaptativo definido como:

$$\rho_t = \alpha \mathcal{N}(x_t \mid \mu_{i,t-1}, \sigma_{i,t-1}^2).$$

Aquí, α actúa como tasa de aprendizaje global del modelo, mientras que ρ_t ajusta dinámicamente cuánto se actualizan la media y la varianza en función de lo bien que la gaussiana explica el valor observado.

Creación y eliminación de gaussianas Si ninguna gaussiana coincide con x_t , se crea una nueva con:

$$\mu_{new} = x_t, \quad \sigma_{new}^2 = \sigma_0^2, \quad w_{new} = w_0.$$

Las gaussianas con peso muy bajo se eliminan automáticamente, lo que permite que K_t se mantenga pequeño y adaptado a la complejidad del píxel.

Selección del fondo Las gaussianas se ordenan según:

$$\frac{w_{i,t}}{\sigma_{i,t}},$$

y las primeras B gaussianas que satisfacen:

$$\sum_{i=1}^B w_{i,t} > T_b$$

constituyen el modelo del fondo. Si x_t coincide con alguna de ellas, se clasifica como fondo; en caso contrario, como primer plano.

Este mecanismo adaptativo permite que *MOG2* ajuste automáticamente la complejidad del modelo, mejorando su estabilidad ante cambios de iluminación, ruido y variaciones dinámicas en la escena.

2.2.2. Ejemplo para comprender la formulación matemática

Se considera un píxel cuyo valor en el instante t es $x_t = 120$ (imagen en escala de grises) y se supone que en el instante anterior $t - 1$ el modelo utilizaba $K_{t-1} = 2$ gaussianas con los siguientes parámetros:

$$\begin{aligned} \text{Gaussiana 1: } & w_{1,t-1} = 0,7, \quad \mu_{1,t-1} = 118, \quad \sigma_{1,t-1}^2 = 4, \\ \text{Gaussiana 2: } & w_{2,t-1} = 0,3, \quad \mu_{2,t-1} = 150, \quad \sigma_{2,t-1}^2 = 9. \end{aligned}$$

1. Evaluación de la probabilidad del píxel La probabilidad total del píxel se calcula mediante el modelo de mezcla:

$$p(x_t) = \sum_{i=1}^2 w_{i,t-1} \mathcal{N}(x_t \mid \mu_{i,t-1}, \sigma_{i,t-1}^2) = w_{1,t-1} \mathcal{N}(120 \mid 118, 4) + w_{2,t-1} \mathcal{N}(120 \mid 150, 9),$$

donde la densidad de una distribución normal unidimensional es:

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Gaussiana 1

$$\mathcal{N}(120 | 118, 4) = \frac{1}{\sqrt{2\pi \cdot 4}} \exp\left(-\frac{(120 - 118)^2}{2 \cdot 4}\right) = \frac{1}{\sqrt{8\pi}} \exp(-0,5) \approx 0,1210.$$

Gaussiana 2

$$\mathcal{N}(120 | 150, 9) = \frac{1}{\sqrt{2\pi \cdot 9}} \exp\left(-\frac{(120 - 150)^2}{2 \cdot 9}\right) = \frac{1}{\sqrt{18\pi}} \exp(-50) \approx 2,5649 \cdot 10^{-23} \approx 0.$$

Probabilidad total del píxel

$$\begin{aligned} p(x_t) &= 0,7 \cdot 0,1210 + 0,3 \cdot 2,5649 \cdot 10^{-23} \\ &\approx 0,7 \cdot 0,1210. \end{aligned}$$

Dado que el segundo término es despreciable:

$$p(x_t = 120) \approx 0,0847.$$

Esto muestra que la primera gaussiana explica casi por completo el valor observado del píxel.

2. Determinación de coincidencia para actualizar la gaussiana correspondiente El píxel coincide con la gaussiana 1 si:

$$|x_t - \mu_{1,t-1}| \leq k \sigma_{1,t-1},$$

con $k \in [2,5; 3]$, que determina cuántas desviaciones estándar se consideran cercanas. De la estadística matemática se sabe que valores dentro de $\pm 3\sigma$ desviaciones estándar cubren el 99.7 % de la masa de una distribución normal.

En este caso:

$$|120 - 118| = 2 \leq 3 \cdot 2,$$

por lo que la gaussiana 1 coincide con el píxel, mientras que la gaussiana 2 no.

3. Actualización de pesos Con $\alpha = 0,05$, los pesos se actualizan como:

$$w_{1,t} = (1 - \alpha)w_{1,t-1} + \alpha \cdot 1, \quad w_{2,t} = (1 - \alpha)w_{2,t-1} + \alpha \cdot 0.$$

4. Actualización de media y varianza Para la gaussiana coincidente:

$$\rho_t = \alpha \mathcal{N}(x_t | \mu_{1,t-1}, \sigma_{1,t-1}^2).$$

Si, por ejemplo, $\rho_t = 0,02$, entonces:

$$\mu_{1,t} = (1 - \rho_t)\mu_{1,t-1} + \rho_t x_t = 0,98 \cdot 118 + 0,02 \cdot 120,$$

$$\sigma_{1,t}^2 = (1 - \rho_t)\sigma_{1,t-1}^2 + \rho_t(x_t - \mu_{1,t})^2.$$

La gaussiana 2 solo reduce su peso y no actualiza media ni varianza.

5. Selección del fondo Las gaussianas se ordenan según el criterio:

$$\frac{w_{i,t}}{\sigma_{i,t}}.$$

A continuación, se calcula el peso acumulado sumando los pesos en ese orden:

$$\sum_{i=1}^B w_{i,t} > T_b,$$

donde B es el número mínimo de gaussianas necesarias para superar el umbral T_b . Estas primeras B gaussianas constituyen el modelo del fondo. El píxel se clasifica como fondo únicamente si coincide con alguna de las gaussianas del conjunto de fondo (las primeras B). Si no coincide con ninguna de ellas, se clasifica como primer plano.

2.2.3. Aprendizaje del fondo en *MOG2*

El modelo *MOG2* no dispone de una imagen de fondo inicial, sino que la aprende de manera progresiva a partir de los primeros fotogramas. La idea fundamental es que los píxeles que permanecen estables durante un periodo prolongado tienden a formar parte del fondo, mientras que los que cambian con frecuencia corresponden al primer plano. Para cada píxel, el modelo mantiene una mezcla de gaussianas y actualiza sus parámetros en función de la estabilidad temporal: las gaussianas que explican valores constantes aumentan su peso $w_{i,t}$ y terminan superando el umbral acumulado T_b , pasando así a formar parte del modelo del fondo.

A este proceso de aprendizaje se le conoce como *warm-up*. Durante los primeros fotogramas se aplica el sustractor de fondo pero sin utilizar todavía la máscara resultante. En esta etapa, el modelo observa la escena y ajusta las gaussianas asociadas a cada píxel. Los elementos estáticos como la carretera, las señales o las marcas viales generan observaciones coherentes que refuerzan las gaussianas correspondientes, mientras que los objetos en movimiento (vehículos) producen valores variables que no llegan a consolidarse. De este modo, al finalizar el *warm-up* el modelo ha aprendido una representación estable del fondo y está preparado para distinguir correctamente entre fondo y primer plano.

Aunque un mayor número de fotogramas de entrenamiento permite obtener un fondo más estable y menos ruidoso, también implica un riesgo: si un objeto del primer plano permanece inmóvil durante un tiempo suficiente, sus observaciones pueden llegar a consolidarse en el modelo del fondo. Esto puede ocurrir, por ejemplo, con vehículos detenidos durante largos periodos o con sombras persistentes, que acaban siendo absorbidos por el fondo debido al aumento de su peso en la mezcla de gaussianas.

Ejemplo ilustrativo del *warm-up*. Para comprender de manera intuitiva el proceso de aprendizaje del fondo en *MOG2*, se considera una secuencia de vídeo captada desde un pórtico sobre una autopista. Durante los primeros instantes, el modelo no dispone de información previa y debe inferir qué píxeles pertenecen al fondo y cuáles corresponden a objetos en movimiento. Para ello, mantiene para cada píxel una mezcla de gaussianas que se actualizan en función de las observaciones temporales. Se asumen 120 fotogramas para el aprendizaje.

Fotograma 1. Un píxel situado sobre el asfalto presenta un valor de intensidad cercano a 80, mientras que un píxel que coincide con un coche blanco toma un valor alrededor de 230. En esta

fase inicial, ambas observaciones se registran, pero ninguna gaussiana posee aún suficiente peso como para ser considerada parte del fondo.

Fotograma 10. El asfalto continúa mostrando valores estables en torno a 80 (80, 82, 79, 81...), mientras que el coche blanco ya no está en ese píxel y su valor cambia a 40 debido al paso de otro vehículo. La gaussiana asociada al asfalto acumula observaciones coherentes y aumenta su peso, mientras que la correspondiente al coche blanco pierde relevancia al no repetirse.

Fotograma 50. El asfalto sigue siendo estable, mientras que los vehículos generan valores muy variables y de corta duración. En este punto, la gaussiana del asfalto supera el umbral acumulado T_b y el píxel pasa a formar parte del modelo del fondo.

Fotograma 120 (fin del *warm-up*). Tras observar suficientes fotogramas, el modelo ha consolidado como fondo aquellos elementos estáticos de la escena: el asfalto, las líneas viales, las señales y el guardarraíl. Por el contrario, los vehículos han producido observaciones inconsistentes que no han logrado estabilizarse en ninguna gaussiana dominante. A partir de este momento, cualquier desviación significativa respecto al modelo del fondo se clasifica como primer plano, generando una máscara de movimiento fiable.

Caso especial: un vehículo detenido. Si un coche permanece inmóvil durante un número elevado de fotogramas, sus observaciones dejan de ser consideradas ruido y comienzan a parecer un valor estable. En consecuencia, la gaussiana correspondiente puede aumentar su peso hasta superar el umbral T_b , provocando que el vehículo sea absorbido por el fondo. Este fenómeno explica por qué un *warm-up* excesivamente largo puede introducir errores en escenas donde existen objetos estáticos prolongados.

2.2.4. Implementación en *OpenCV*

Aunque el modelo matemático de *MOG2* define explícitamente la actualización de pesos, medias, varianzas y el criterio de selección del fondo, la librería utilizada, *OpenCV* [9], no expone estos parámetros de forma directa. En su lugar, la función `cv2.createBackgroundSubtractorMOG2` gestiona internamente:

- la tasa de aprendizaje global α ,
- el factor adaptativo ρ_t ,
- el número dinámico de gaussianas K_t ,
- el criterio de coincidencia basado en desviaciones estándar,
- la creación y eliminación de gaussianas,
- y la selección del fondo mediante el peso acumulado.

El usuario solo controla parámetros de alto nivel como `history` y `varThreshold`, calculando la tasa de aprendizaje global de la siguiente manera:

$$\alpha = \frac{1}{\text{history}},$$

El parámetro `varThreshold` controla el criterio de coincidencia entre el valor observado de un píxel y cada una de las gaussianas del modelo. En la implementación de *OpenCV*, un píxel se considera explicado por una gaussiana si su distancia al cuadrado respecto a la media satisface

$$\|x_t - \mu_k\|^2 < \text{varThreshold} \cdot \sigma_k^2,$$

lo que equivale al criterio teórico $|x_t - \mu_k| \leq k \sigma_k$ con `varThreshold` = k^2 . De este modo, `varThreshold` determina cuántas desviaciones estándar se permiten para considerar que un píxel pertenece al fondo: valores bajos hacen el modelo más estricto, mientras que valores altos permiten mayor variabilidad. En este trabajo se utiliza `varThreshold` = 25, correspondiente a $k = 5$, lo que proporciona una mayor tolerancia frente a pequeñas variaciones de iluminación y ruido en escenas de tráfico.

En cuanto al umbral T_b utilizado en la formulación teórica para decidir qué gaussianas forman el modelo del fondo, *OpenCV* también lo gestiona de manera interna, no es accesible desde la *API* y se mantiene fijo en 0,7, es decir, el fondo está formado por las primeras gaussianas cuyo peso acumulado supere el 70 %.

2.2.5. Resultados

En las figuras 1, 2 y 3 se presenta el resultado de aplicar la sustracción del fondo mediante *Mog2*. En la segunda figura se observa la presencia de ruido, aspecto fundamental a tener en cuenta para evitar detectar falsos vehículos.

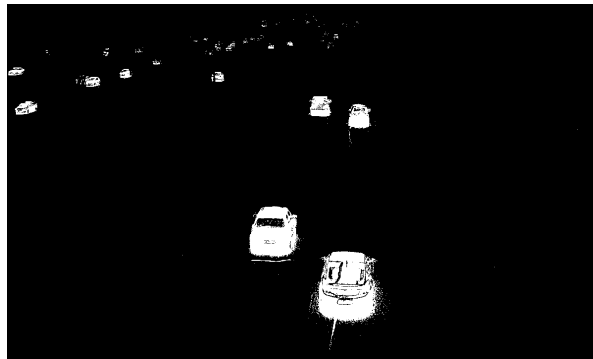


Figura 1: Sustracción del fondo con *Mog2*.



Figura 2: Sustracción ruidosa del fondo con *Mog2*.

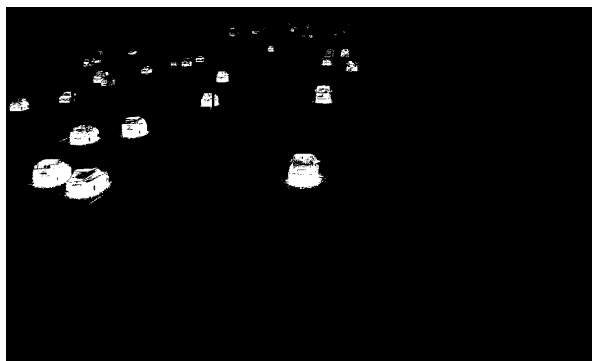


Figura 3: Sustracción del fondo con *Mog2* con dos vehículos cercanos.

2.3. Extracción de regiones y detección de vehículos

Una vez obtenida la máscara de movimiento mediante el sustractor de fondo *MOG2*, se aplica un proceso de filtrado morfológico destinado a reducir el ruido inherente a la supresión del fondo, figura 2. En primer lugar, se realiza una apertura morfológica que elimina píxeles aislados y pequeñas regiones espurias, figura 4. A continuación, se aplican operaciones de dilatación y cierre que permiten reconstruir las regiones correspondientes a los vehículos y rellenar posibles huecos internos, evitando perder el seguimiento de un vehículo en algún instante que no mantenga el identificador del vehículo, ya que si se vuelve a detectar después se le asigna un identificador diferente.

Sin embargo, las operaciones de dilatación y cierre introducen problemas de segmentación cuando el tráfico es denso, figura 5, es decir, varios vehículos cercanos los reconoce como uno único. Si se eliminan estas operaciones, figura 6, se soluciona, pero pueden aparecer pérdidas de seguimiento como ya se ha mencionado, figura 7, que se trata en la sección 3.2.

Para solucionar estos problemas de segmentación existen técnicas más avanzadas como el análisis de forma o la detección de bordes internos mediante Canny, utilizada en métodos de segmentación basados en procesamiento de imagen [2], la utilización de técnicas heurísticas [1], la aplicación de contornos deformables a los vehículos [10], la agrupación de puntos característicos estables [1] o la segmentación basada en patrones de movimiento apoyada en técnicas de flujo óptico [1, 11, 12].

No obstante, al tratarse de la segunda tarea evaluable de una asignatura de 6 créditos ECTS, por simplicidad se ha decidido no abordar estas técnicas.

Posteriormente, se procede a la extracción de contornos mediante *findContours* de la librería de *OpenCV*. Para cada región conectada se calcula un rectángulo envolvente (*bounding box*), siempre que su área supere un umbral mínimo que permite descartar ruido y pequeñas fluctuaciones no asociadas a vehículos.

Cada uno de estos rectángulos se interpreta como una detección candidata de vehículo. Este enfoque basado en *blobs*, es decir, en regiones conectadas de píxeles etiquetados como primer plano tras la sustracción de fondo, es ampliamente utilizado en sistemas de monitorización de tráfico [1], debido a su simplicidad y eficiencia computacional. No obstante, presenta desafíos de segmentación como ya se ha expuesto.

2.4. Región de interés

Para facilitar la detección de los vehículos y evitar falsos positivos en regiones pequeñas de la imagen se ha seleccionado una región de interés (ROI) mediante una interfaz gráfica, figura 8. Esta región permite descartar automáticamente todos aquellos píxeles que se encuentren fuera de ella mediante una máscara binaria.

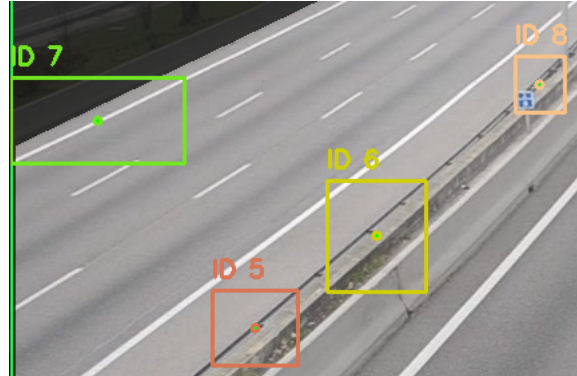


Figura 4: Píxeles aislados o regiones espurias.

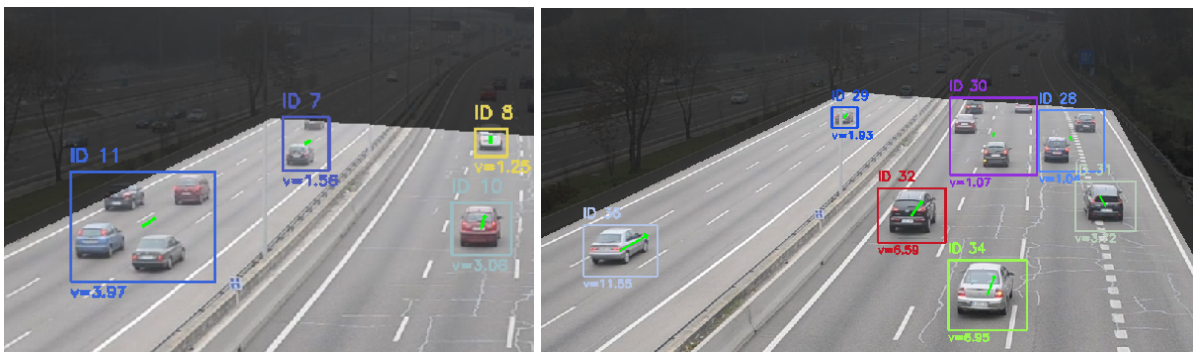


Figura 5: Problemas de segmentación.

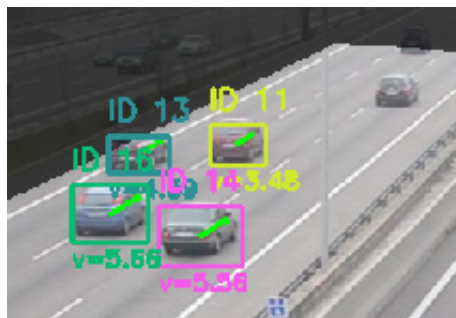


Figura 6: Problemas de segmentación solucionados.

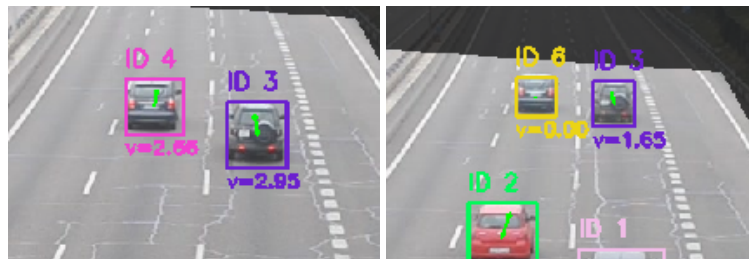


Figura 7: Problemas de seguimiento (ID 4 se transforma en ID 6).

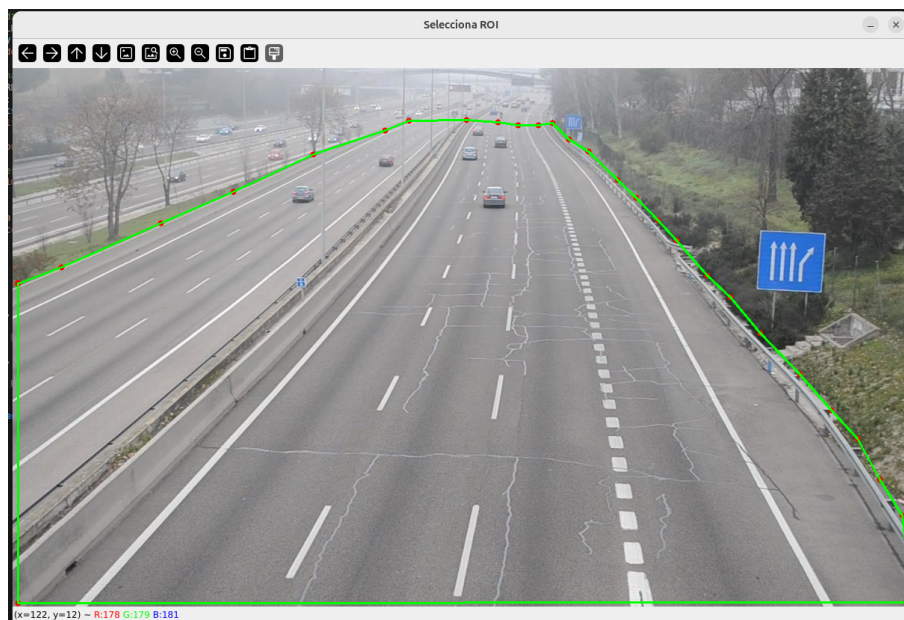


Figura 8: Región de interés seleccionada.

3. Flujo óptico

El flujo óptico describe el campo de movimiento aparente entre dos instantes de tiempo consecutivos. Matemáticamente, se basa en la suposición de que la intensidad luminosa de un píxel se mantiene aproximadamente constante a lo largo del tiempo mientras se desplaza en la imagen.

En este proyecto se utiliza el método denso de Farnebäck [5], una técnica de estimación densa que aproxima la intensidad local de la imagen mediante una expansión polinómica de segundo grado y que proporciona un vector de flujo (v_x, v_y) para cada píxel de la imagen. Dado un par de *frames* en escala de grises, se calcula el flujo óptico y para cada vehículo detectado se obtiene un vector de flujo medio dentro de su región. Este vector representa la dirección y la magnitud predominante del movimiento del vehículo en dicho fotograma.

En cuanto a los métodos estudiados en la asignatura, Lucas–Kanade y Gauss–Seidel presentan limitaciones importantes en el contexto de la monitorización de tráfico. Lucas–Kanade [13] calcula el flujo óptico de forma esparsa, únicamente en puntos con suficiente textura, lo que resulta problemático en vehículos con superficies homogéneas, bajo contraste o sometidos a oclusiones parciales. Por su parte, los métodos iterativos basados en Gauss–Seidel tienden a generar campos de flujo excesivamente suavizados, lo que dificulta capturar variaciones locales del movimiento en regiones amplias y poco texturizadas.

El enfoque denso de Farnebäck ofrece una mayor robustez frente a ruido, sombras y variaciones de iluminación, y permite obtener un vector de movimiento medio dentro de cada región asociada a un vehículo. Tal como señalan trabajos previos en visión aplicada al tráfico [14], los métodos esparsos pueden fallar en vehículos con poca textura, mientras que los métodos densos permiten una estimación más estable del movimiento global del objeto, lo que justifica la elección de Farnebäck en este sistema.

3.1. Formulación matemática del flujo óptico de Farnebäck

El flujo óptico describe el campo de desplazamientos aparentes de los píxeles entre dos fotogramas consecutivos de una secuencia de vídeo. Bajo la hipótesis de constancia de brillo, se asume que la intensidad de un punto de la escena se mantiene aproximadamente constante entre dos instantes temporales:

$$I(x, y, t) \approx I(x + u, y + v, t + 1),$$

donde:

- $I(x, y, t)$ es la intensidad del píxel en la posición (x, y) en el instante t ,
- (u, v) es el desplazamiento del punto entre los instantes t y $t + 1$, es decir, el vector de flujo óptico.

Desarrollando en serie de Taylor la intensidad en el instante $t + 1$ alrededor de (x, y, t) y despreciando términos de orden superior, se obtiene la ecuación clásica de restricción del flujo óptico [15]:

$$I_x u + I_y v + I_t = 0,$$

donde I_x , I_y e I_t son las derivadas parciales de la intensidad con respecto a x , y y t , respectivamente. Esta ecuación es insuficiente por sí sola para determinar (u, v) , ya que aporta una única ecuación para dos incógnitas.

El método de Farnebäck [5] propone aproximar la intensidad local de la imagen en una vecindad mediante un polinomio de segundo grado:

$$I(x) \approx x^\top A x + b^\top x + c,$$

donde:

- $x = \begin{bmatrix} x \\ y \end{bmatrix}$ es el vector de coordenadas espaciales,
- A es una matriz simétrica 2×2 que captura la curvatura local,
- b es un vector 2×1 que representa el gradiente lineal,
- c es un escalar que representa el término constante.

Si se asume que entre dos instantes consecutivos la intensidad local se conserva y sólo cambia la posición del patrón de imagen, se puede escribir:

$$I_1(x) \approx x^\top A x + b^\top x + c,$$

$$I_2(x) \approx (x - d)^\top A (x - d) + b^\top (x - d) + c,$$

donde I_1 e I_2 son las intensidades en los instantes t y $t + 1$, respectivamente, y $d = \begin{bmatrix} u \\ v \end{bmatrix}$ es el desplazamiento desconocido que se desea estimar.

Al expandir la expresión de $I_2(x)$ se obtiene:

$$I_2(x) \approx x^\top A x + (b - 2Ad)^\top x + (d^\top A d - b^\top d + c).$$

Comparando los coeficientes de los polinomios que aproximan I_1 e I_2 , se observa que la matriz A se mantiene aproximadamente constante, mientras que el término lineal cambia de b a $b' = b - 2Ad$. Esta relación permite expresar el desplazamiento d en función de A y de la diferencia entre los términos lineales de ambos instantes:

$$b' \approx b - 2Ad \quad \Rightarrow \quad 2Ad \approx b - b'.$$

Si se dispone de múltiples estimaciones locales de (A, b, b') en una vecindad alrededor de un punto, es posible plantear un sistema sobredeterminado y estimar el desplazamiento d en el sentido de mínimos cuadrados. En términos generales, el problema adopta la forma:

$$Md \approx e,$$

donde M y e se construyen a partir de las contribuciones locales de los términos polinómicos, y la solución estimada viene dada por:

$$d = (M^T M)^{-1} M^T e.$$

El resultado final es un campo de flujo óptico denso, en el que a cada píxel se le asocia un vector:

$$d(x, y) = \begin{bmatrix} u(x, y) \\ v(x, y) \end{bmatrix},$$

que representa el desplazamiento en píxeles entre los dos fotogramas considerados.

Aunque la formulación matemática de Farnebäck describe explícitamente la aproximación polinómica local, la comparación de coeficientes entre fotogramas y la resolución del sistema sobredeterminado para obtener el desplazamiento d , la implementación práctica en *OpenCV*, sección 3.4, abstrae estos pasos internos.

La función `cv2.calcOpticalFlowFarneback` no expone directamente las matrices A , los vectores b y b' ni la construcción explícita del sistema $Md \approx e$. En su lugar, permite controlar estos elementos de forma indirecta mediante parámetros de alto nivel que regulan el tamaño de las vecindades polinómicas, el suavizado gaussiano previo, la resolución de la pirámide y el número de iteraciones empleadas para refinar la estimación.

De este modo, cada parámetro de la implementación corresponde a un aspecto concreto del modelo teórico, pero sin requerir que el usuario implemente manualmente la expansión polinómica ni la resolución del sistema de mínimos cuadrados.

Es importante señalar que la formulación matemática original del método de Farnebäck no incluye el uso de una pirámide de imágenes. La pirámide es una estrategia computacional añadida en implementaciones prácticas, como la de *OpenCV*, para mejorar la robustez del algoritmo frente a movimientos grandes entre fotogramas. Consiste en generar versiones de la imagen a distintas resoluciones y estimar el flujo óptico de forma jerárquica, comenzando en la resolución más baja, donde los desplazamientos aparentes son más pequeños, y refinando progresivamente la estimación en niveles de mayor detalle. Este enfoque multiescala no altera el modelo polinómico descrito por Farnebäck, sino que actúa como un mecanismo de estabilización y permite capturar los movimientos grandes con menos errores.

Una ilustración de estas pirámides puede verse en la figura 9.

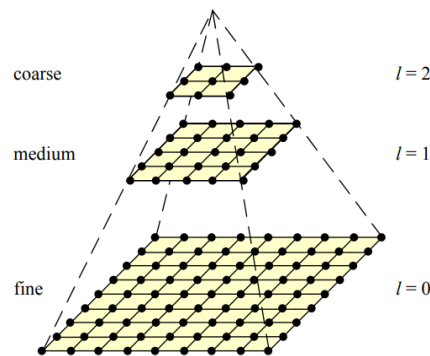


Figura 9: Representación multiresolución en pirámide de imágenes [16].

3.1.1. Ejemplo ilustrativo del vector de flujo óptico

Para ilustrar el significado del vector de flujo óptico, se considera un objeto que se desplaza horizontalmente hacia la derecha en la imagen. Supóngase que su centroide se encuentra en la posición $(x_t, y_t) = (100, 50)$ en el instante t y en la posición $(x_{t+1}, y_{t+1}) = (104, 50)$ en el instante $t + 1$. El desplazamiento observado es:

$$u = x_{t+1} - x_t = 104 - 100 = 4, \quad v = y_{t+1} - y_t = 50 - 50 = 0.$$

En este caso, el flujo óptico medio asociado a la región del objeto vendrá dado por el vector:

$$d = \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \end{bmatrix} \text{ px/frame.}$$

Si la frecuencia del vídeo es de $f = 25$ fotogramas por segundo, el módulo de la velocidad en píxeles por segundo sería:

$$\|v_{\text{px/s}}\| = \sqrt{u^2 + v^2} \cdot f = \sqrt{4^2 + 0^2} \cdot 25 = 4 \cdot 25 = 100 \text{ px/s.}$$

En el sistema desarrollado, en lugar de trabajar con la norma del vector, se emplea una medida de velocidad relativa en píxeles por fotograma basada en el módulo del vector de flujo óptico:

$$v_{\text{px/frame}} = \sqrt{u^2 + v^2},$$

o, de forma aproximada, mediante una combinación lineal de sus componentes (habitualmente $|u| + |v|$), lo que permite comparar el movimiento de distintos vehículos sin necesidad de realizar una calibración geométrica de la cámara.

3.2. Seguimiento de objetos

El seguimiento de vehículos se implementa mediante un esquema sencillo basado en centroides. Cada vehículo detectado se representa como un objeto seguido con un identificador único, un centroide, una caja envolvente (*bounding box*) y un estado interno que contiene el número de fotogramas consecutivos en los que no ha sido detectado y el vector medio de flujo óptico.

En cada *frame* se sigue el siguiente procedimiento:

1. Se obtienen las detecciones actuales (cajas y centroides) a partir de la máscara de movimiento.
2. Para cada detección, se busca el objeto seguido cuyo centroide anterior esté más próximo al centroide actual, siempre que la distancia euclídea sea menor que un umbral predefinido. Habrá que prestar especial atención a este umbral, ya que un umbral de distancia demasiado estricto o demasiado laxo puede generar problemas de seguimiento.
3. Si se encuentra una correspondencia, se actualiza la posición del objeto seguido con la nueva detección y se reinicia su contador de pérdida.
4. Si no se encuentra correspondencia, se crea un nuevo objeto seguido con un identificador nuevo.
5. Los objetos que no han sido actualizados durante un número máximo de frames se eliminan del sistema. También será necesario prestar especial atención a este parámetro para minimizar errores en el seguimiento, por ejemplo, el problema expuesto en la figura 7.

Además, para cada objeto seguido se calcula el vector medio de flujo óptico dentro de su *bounding box*. Este vector se almacena como parte del estado del objeto y se utiliza para visualizar una flecha sobre el centroide del vehículo, indicando así la dirección media de su movimiento.

3.3. Cálculo de la velocidad de movimiento del objeto

El sistema de seguimiento implementado calcula la velocidad de cada vehículo a partir del flujo óptico, obteniendo un vector de movimiento medio dentro de la región correspondiente al vehículo [17]. Este vector se expresa en *píxeles por fotograma* (px/frame). Sin embargo, esta magnitud no tiene una interpretación física directa, ya que la distancia en píxeles no corresponde a una distancia real en metros.

Para convertir una velocidad expresada en píxeles por fotograma a una velocidad real en kilómetros por hora, por ejemplo, es necesario conocer la relación entre píxeles y metros en la escena. Esta relación depende de la geometría de la cámara y del entorno y se denomina factor de escala. En términos generales, la conversión se realiza mediante:

$$v_{\text{real}} = v_{\text{px}} \cdot S \cdot f,$$

donde:

- v_{px} es la velocidad medida en píxeles por fotograma,
- S es el factor de escala (metros por píxel),
- f es la frecuencia de captura del vídeo (fotogramas por segundo).

Una vez obtenida la velocidad en metros por segundo, la conversión a kilómetros por hora se realiza mediante:

$$v_{\text{km/h}} = v_{\text{m/s}} \cdot 3,6.$$

El problema fundamental es que el factor de escala S no es constante en una escena real con perspectiva. En una carretera vista desde una cámara elevada, los objetos cercanos ocupan más píxeles que los objetos lejanos, por lo que la relación metros/píxel varía con la posición en la imagen [14]. Por tanto, para calcular S es necesario conocer parámetros como la altura de la cámara, la distancia a la carretera, el ángulo de inclinación, la matriz intrínseca de la cámara y un modelo de homografía [18] entre la imagen y el plano de la carretera [19].

Por tanto, para obtener un factor de escala S exacto en toda la imagen es necesario calibrar la cámara y estimar la homografía entre la imagen y el plano de la carretera. Esta homografía se deriva de la geometría de cámara *pinhole*, figura 10 y se obtiene a partir de los parámetros intrínsecos y extrínsecos de la cámara.

El modelo *pinhole* representado en la figura 10 describe la geometría de proyección en un plano de imagen continuo, donde la distancia focal f es una magnitud física única. Sin embargo, en una imagen digital las coordenadas del plano de imagen se expresan en píxeles, no en unidades métricas. Esta discretización introduce factores de conversión distintos en las direcciones horizontal y vertical, lo que da lugar a las distancias focales en píxeles f_x y f_y , así como al punto principal (c_x, c_y) . Estos parámetros ópticos de la cámara se agrupan en la matriz intrínseca K [20], que permite relacionar el modelo geométrico ideal con las coordenadas discretas de la imagen:

$$K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix},$$

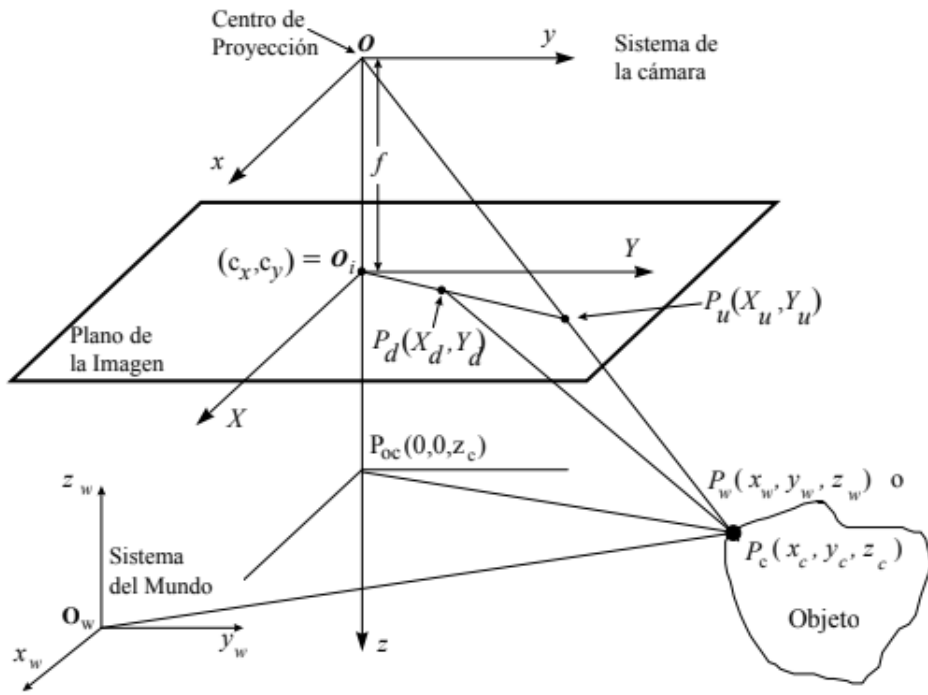


Figura 10: Modelo de cámara geométrica ideal *pinhole*. Fuente: Campus Virtual.

donde f_x y f_y son las distancias focales en píxeles y (c_x, c_y) es el punto principal. Estos parámetros se obtienen mediante calibración interna o a partir de los metadatos de la cámara ya que en caso de conocer p_x y p_y , es decir, el tamaño físico del píxel (mm/píxel) en cada eje, se pueden calcular las distancias focales expresadas en píxeles mediante $f_x = f/p_x$ y $f_y = f/p_y$. No obstante, el procedimiento más común es la calibración.

La matriz de rotación R describe la orientación de la cámara respecto al mundo. Incluye el ángulo de inclinación (*pitch*), así como posibles componentes de *roll* y *yaw*. Se calcula a partir de la orientación física de la cámara o mediante métodos automáticos basados en puntos de fuga [19].

El vector de traslación t indica la posición de la cámara en el sistema de coordenadas del mundo. Su componente vertical corresponde a la altura de la cámara sobre la carretera, que puede medirse directamente o estimarse mediante calibración geométrica.

El plano de la carretera se modela mediante la ecuación:

$$n^T X + d = 0,$$

donde n es el vector normal del plano y d es la distancia desde el origen al plano. Para una carretera plana horizontal, típicamente $n = (0, 1, 0)^T$ y d coincide con la altura de la cámara.

Con estos parámetros, la homografía entre la imagen y el plano de la carretera viene dada por:

$$H = K \left(R - \frac{t n^T}{d} \right) K^{-1}.$$

Esta expresión incorpora explícitamente la distancia focal, el punto principal, la orientación de la cámara, su altura y la geometría del plano de la carretera. Una vez conocida la homografía,

cualquier punto de la imagen $\mathbf{x} = (u, v, 1)^\top$ puede transformarse en coordenadas métricas del plano mediante:

$$\mathbf{X} = H^{-1}\mathbf{x}.$$

El factor de escala en una posición concreta de la imagen se obtiene evaluando el desplazamiento real inducido por un desplazamiento unitario en la imagen:

$$S(\mathbf{x}) = \|H^{-1}\Delta\mathbf{x}\|,$$

lo que proporciona la relación metros/píxel exacta en cada punto de la imagen. Este enfoque permite corregir completamente la distorsión debida a la perspectiva y constituye la base de los métodos modernos de calibración para monitorización de tráfico [19].

Dado que estos parámetros no están disponibles en el *dataset* utilizado, no es posible obtener una conversión fiable de píxeles/frame a km/h. Por este motivo, en este proyecto la velocidad se expresa únicamente en unidades relativas (píxeles por fotograma), que permiten comparar el movimiento entre vehículos, pero no obtener una velocidad física real.

También existen métodos alternativos como el propuesto en [21], que no utiliza una homografía completa entre la imagen y el plano de la carretera. En su lugar, los autores emplean un modelo geométrico simplificado para convertir el desplazamiento en píxeles de un vehículo en una distancia real aproximada.

El procedimiento comienza midiendo el desplazamiento del vehículo entre dos fotogramas consecutivos mediante flujo óptico. A partir de este desplazamiento en píxeles, se obtiene una velocidad inicial expresada en píxeles por fotograma. Para convertir esta magnitud en una distancia física, el método asume que la carretera es un plano y que la cámara está situada a una altura conocida h , con una distancia focal f y un ángulo de inclinación θ . Bajo estas hipótesis, la posición vertical del vehículo en la imagen y se relaciona con su distancia real d mediante la proyección perspectiva:

$$y = f \cdot \frac{h}{d \cos \theta - h \sin \theta}.$$

Invirtiendo esta expresión se obtiene una estimación de la distancia del vehículo a la cámara en función de su posición en la imagen:

$$d(y) = \frac{fh}{y \cos \theta} + h \tan \theta.$$

El factor de escala local, que expresa cuántos metros reales corresponden a un desplazamiento de un píxel en la vecindad del vehículo, se obtiene derivando la distancia respecto a la coordenada vertical:

$$S_{\text{local}}(y) = \left| \frac{\partial d}{\partial y} \right| = \frac{fh}{y^2 \cos \theta}.$$

Este factor de escala no constituye una homografía global, sino una aproximación válida únicamente en la vecindad del vehículo. Dado que el desplazamiento entre dos fotogramas consecutivos es pequeño, el valor de $S_{\text{local}}(y)$ puede considerarse constante en ese intervalo, permitiendo convertir un desplazamiento en píxeles Δy en una distancia real aproximada:

$$\Delta d \approx S_{\text{local}}(y) \Delta y.$$

Por tanto, el método permite estimar la velocidad real del vehículo sin necesidad de una calibración completa de la cámara, aunque no resuelve la variación global del factor de escala debida a la perspectiva en toda la imagen [21].

3.4. Implementación en *OpenCV*

Siguiendo la explicación al final de la sección 3.1, se describen los parámetros utilizados y la justificación de los valores adoptados en la implementación de *OpenCV*:

- `pyr_scale`: factor de reducción entre niveles de la pirámide. Valores típicos oscilan entre 0.3 y 0.8. Un valor de 0.5 reduce cada nivel a la mitad de resolución, permitiendo capturar movimientos grandes sin perder estabilidad. En este trabajo se adopta `pyr_scale = 0.5`, ya que los vehículos pueden desplazarse varios píxeles entre fotogramas.
- `levels`: número de niveles de la pirámide. Más niveles permiten detectar movimientos mayores, pero incrementan el coste computacional. Se emplean entre 3 y 5 niveles en aplicaciones de tráfico. En este trabajo se utiliza `levels = 3`, suficiente para capturar movimientos moderados manteniendo un tiempo de cómputo reducido.
- `winsize`: tamaño de la ventana para la expansión polinómica. Ventanas pequeñas (9–13 píxeles) son sensibles al ruido, mientras que ventanas grandes (21–31) producen un flujo más suave pero menos detallado. En este sistema se utiliza `winsize = 15`, que ofrece un equilibrio adecuado entre suavidad y capacidad para seguir contornos de vehículos.
- `iterations`: número de iteraciones por nivel de la pirámide. Más iteraciones permiten refinar el flujo, pero aumentan el tiempo de cálculo. Valores entre 1 y 5 son habituales. Se adopta `iterations = 3`, lo que proporciona estabilidad sin penalizar excesivamente el rendimiento.
- `poly_n`: tamaño de la vecindad utilizada para ajustar el polinomio cuadrático local. Valores típicos son 5 o 7. Un valor mayor produce un flujo más suave y robusto, pero puede perder detalles finos. En este trabajo se emplea `poly_n = 7`, adecuado para superficies homogéneas como carrocerías de vehículos.
- `poly_sigma`: desviación típica del suavizado gaussiano aplicado antes del ajuste polinómico. Controla el grado de filtrado del ruido. Valores entre 1.1 y 1.5 son habituales. Se utiliza `poly_sigma = 1.2`, que reduce el ruido sin eliminar variaciones relevantes del flujo.

3.5. Resultados

En la figura 11 se presenta el resultado del seguimiento de vehículos y el cálculo de velocidad con la visualización de la dirección de desplazamiento.

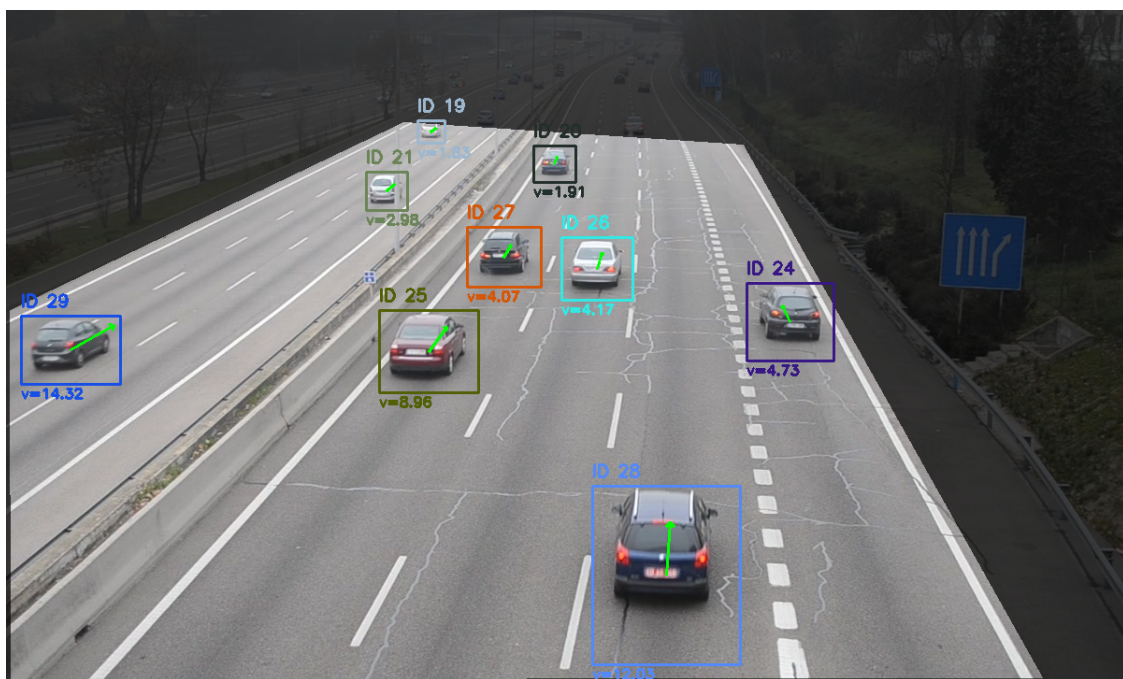


Figura 11: Seguimiento de vehículos, cálculo de velocidad y dirección de desplazamiento.

4. Conclusiones

El sistema desarrollado demuestra que es posible realizar seguimiento de vehículos en una escena de tráfico mediante la combinación de diversas técnicas clásicas de visión por computador, es decir, sin necesidad de técnicas de inteligencia artificial. En particular, la sustracción de fondo permite obtener una estimación inicial del movimiento en la escena, mientras que el flujo óptico proporciona información más detallada sobre la dirección y la velocidad relativa de los vehículos. La integración de ambos métodos, junto con un esquema sencillo de asociación basado en centroides, permite construir un sistema funcional capaz de detectar y seguir vehículos en secuencias de vídeo con un número moderado de objetos y condiciones relativamente controladas.

Este proyecto pone de manifiesto que el desarrollo de aplicaciones completas en visión por computador requiere la combinación de múltiples técnicas procedentes de distintos bloques temáticos. En este caso, se han integrado conceptos de la mayoría de los temas estudiados en la asignatura. La interacción entre estos métodos resulta esencial para obtener un sistema coherente y robusto, y refleja el carácter multidisciplinar de los problemas reales en visión artificial.

A pesar de su simplicidad, el algoritmo de seguimiento empleado resulta adecuado para escenarios sin oclusiones prolongadas ni tráfico denso. No obstante, el sistema presenta limitaciones inherentes a la segmentación basada en movimiento y al uso de un modelo de asociación puramente geométrico, lo que puede provocar pérdidas de seguimiento o intercambios de identidad en situaciones complejas.

Como líneas de trabajo futuro, sería interesante incorporar métodos de seguimiento más sofisticados, como filtros de Kalman y algoritmos de asignación óptima [22] o *trackers* basados en aprendizaje profundo [23]. Asimismo, la sustitución de la detección basada en sustracción de fondo por detectores específicos de vehículos como por ejemplo, redes neuronales convolucionales, esto es, inteligencia artificial, permitiría mejorar la robustez del sistema ante oclusiones, variaciones de iluminación y escenas de tráfico más exigentes, así como una mayor capacidad de adaptación al entorno dinámico. La combinación de estas técnicas avanzadas abriría la puerta a un sistema de monitorización más preciso, escalable y aplicable a escenarios reales.

Además, el desarrollo del sistema pone de manifiesto la importancia de disponer de librerías consolidadas como *OpenCV*, que integran implementaciones eficientes de numerosos métodos clásicos de visión por computador. Gracias a estas herramientas, es posible centrarse en el diseño y la integración del sistema sin necesidad de implementar desde cero algoritmos complejos como la sustracción de fondo, el cálculo del flujo óptico o las operaciones morfológicas. Esto no solo acelera el proceso de desarrollo, sino que garantiza un comportamiento estable y optimizado, permitiendo que el esfuerzo se dirija a la experimentación, el análisis y la comprensión de los métodos estudiados en la asignatura.

Referencias adicionales al material del Campus Virtual

- [1] Neeraj K. Kanhere y Stanley T. Birchfield. «Real-Time Incremental Segmentation and Tracking of Vehicles at Low Camera Angles Using Stable Features». En: *IEEE Transactions on Intelligent Transportation Systems* 9.1 (2008), págs. 148-160. DOI: 10.1109/TITS.2007.911357.
- [2] Prem Kumar Bhaskar y Suet-Peng Yong. «Image processing based vehicle detection and tracking method». En: *2014 International Conference on Computer and Information Sciences (ICCOINS)*. 2014, págs. 1-5. DOI: 10.1109/ICCOINS.2014.6868357.
- [3] Ricardo Guerrero-Gómez-Olmedo et al. «Vehicle Tracking by Simultaneous Detection and Viewpoint Estimation». En: *Natural and Artificial Computation in Engineering and Medical Applications*. Ed. por José Manuel Ferrández Vicente et al. URL: <https://gram.web.uah.es/data/datasets/rtm/index.html>. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, págs. 306-316. ISBN: 978-3-642-38622-0.
- [4] C. Stauffer y W.E.L. Grimson. «Adaptive background mixture models for real-time tracking». En: *Proceedings. 1999 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (Cat. No. PR00149)*. Vol. 2. 1999, 246-252 Vol. 2. DOI: 10.1109/CVPR.1999.784637.
- [5] Gunnar Farnebäck. «Two-Frame Motion Estimation Based on Polynomial Expansion». En: *Image Analysis*. Ed. por Josef Bigun y Tomas Gustavsson. Berlin, Heidelberg: Springer Berlin Heidelberg, 2003, págs. 363-370. ISBN: 978-3-540-45103-7.
- [6] R.J. Radke et al. «Image change detection algorithms: a systematic survey». En: *IEEE Transactions on Image Processing* 14.3 (2005), págs. 294-307. DOI: 10.1109/TIP.2004.838698.
- [7] Z. Zivkovic. «Improved adaptive Gaussian mixture model for background subtraction». En: *Proceedings of the 17th International Conference on Pattern Recognition, 2004. ICPR 2004*. Vol. 2. 2004, 28-31 Vol.2. DOI: 10.1109/ICPR.2004.1333992.
- [8] Zoran Zivkovic y Ferdinand Van der Heijden. «Efficient adaptive density estimation per image pixel for the task of background subtraction». En: *Pattern Recognition Letters* 27.7 (2006), págs. 773-780.
- [9] G. Bradski. «The OpenCV Library». En: *Dr. Dobb's Journal of Software Tools* (2000). URL: <https://opencv.org/>.
- [10] Dieter Koller, Joseph Weber y Jitendra Malik. «Robust multiple car tracking with occlusion reasoning». En: *Proceedings of the Third European Conference on Computer Vision (Vol. 1)*. ECCV '94. Stockholm, Sweden: Springer-Verlag, 1994, págs. 189-196. ISBN: 3540579567.
- [11] Thomas Brox y Jitendra Malik. «Object segmentation by long-term analysis of point trajectories». En: *European Conference on Computer Vision (ECCV)*. Springer, 2010, págs. 282-295.
- [12] Peter Ochs, Jitendra Malik y Thomas Brox. «Segmentation of Moving Objects by Long Term Video Analysis». En: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36.6 (2014), págs. 1187-1200. DOI: 10.1109/TPAMI.2013.242.

- [13] Bruce D. Lucas y Takeo Kanade. «An Iterative Image Registration Technique with an Application to Stereo Vision». En: *Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI)*. 1981, págs. 674–679.
- [14] Benjamin Coifman et al. «A real-time computer vision system for vehicle tracking and traffic surveillance». En: *Transportation Research Part C: Emerging Technologies* 6.4 (1998), págs. 271–288. ISSN: 0968-090X. DOI: [https://doi.org/10.1016/S0968-090X\(98\)00019-9](https://doi.org/10.1016/S0968-090X(98)00019-9). URL: <https://www.sciencedirect.com/science/article/pii/S0968090X98000199>.
- [15] Berthold K. P. Horn y Brian G. Schunck. «Determining optical flow». En: *Artificial Intelligence* 17.1–3 (1981), págs. 185–203.
- [16] Richard Szeliski. *Computer Vision: Algorithms and Applications*. 2nd. Springer, 2022. ISBN: 978-3-030-34372-9.
- [17] F.W. Cathey y D.J. Dailey. «A novel technique to dynamically measure vehicle speed using uncalibrated roadway cameras». En: *IEEE Proceedings. Intelligent Vehicles Symposium, 2005*. 2005, págs. 777–782. DOI: 10.1109/IVS.2005.1505199.
- [18] Enrique Alegre, Gonzalo Pajares y Arturo de la Escalera, eds. *Conceptos y Métodos en Visión por Computador*. CEA (Comité Español de Automática), 2016. URL: <https://www.dropbox.com/scl/fi/nx4xv0m7a3gsrgg/ConceptosyMetodosenVxC-1.pdf>.
- [19] Marketa Dubska, Adam Herout y Jakub Sochor. «Automatic Camera Calibration for Traffic Understanding». En: *Proceedings of the British Machine Vision Conference*. BMVA Press, 2014. DOI: <http://dx.doi.org/10.5244/C.28.42>.
- [20] OpenCV Team. *OpenCV Documentation: Camera Calibration and 3D Reconstruction (calib3d)*. https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html. Accessed: 2026-01-11. 2024.
- [21] Xiao-Chen He y Nelson Hong Ching Yung. «A Novel Algorithm for Estimating Vehicle Speed from Two Consecutive Images». En: *2007 IEEE Workshop on Applications of Computer Vision (WACV '07)* (2007), págs. 12–12. URL: <https://api.semanticscholar.org/CorpusID:2610842>.
- [22] Alex Bewley et al. «Simple online and realtime tracking». En: *IEEE International Conference on Image Processing (ICIP)*. 2016, págs. 3464–3468.
- [23] Wen Luo, Xiaohui Yang y Bo Tong. «Fast vehicle detection in traffic surveillance using convolutional neural networks». En: *IEEE Access* 6 (2018), págs. 73795–73805.